

Университет ИТМО

Факультет программной инженерии и компьютерной техники
Образовательная программа системное и прикладное
программное обеспечение

Лабораторная работа №5
По дисциплине "Вычислительная математика"
Вариант №2

Выполнил студент группы Р3209
Евграфов Артём Андреевич
Проверил:
Рыбаков Степан Дмитриевич

Санкт-Петербург 2026

Содержание

1. Вычислительная реализация	2
1.1. Исходные данные (Вариант 2)	2
1.2. Таблица конечных разностей	2
1.3. Вычисление значения функции для $X_1 = 0,502$	2
1.4. Вычисление значения функции для $X_2 = 0,645$	2
1.5. Вычисление значений через многочлен Лагранжа	3
2. Листинг программы	4
3. Вывод	9

1. Вычислительная реализация

1.1. Исходные данные (Вариант 2)

Задана таблица значений функции $y = f(x)$:

i	0	1	2	3	4	5	6
x_i	0,50	0,55	0,60	0,65	0,70	0,75	0,80
y_i	1,5320	2,5356	3,5406	4,5462	5,5504	6,5559	7,5594

Точки для интерполирования: $X_1 = 0,502$, $X_2 = 0,645$.

Шаг таблицы: $h = 0,05$.

1.2. Таблица конечных разностей

Вычислим конечные разности $\Delta^k y_i = \Delta^{k-1} y_{i+1} - \Delta^{k-1} y_i$:

i	x_i	y_i	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$	$\Delta^5 y$	$\Delta^6 y$
0	0,50	1,5320	1,0036	0,0014	-0,0008	-0,0012	0,0059	-0,0166
1	0,55	2,5356	1,0050	0,0006	-0,0020	0,0047	-0,0107	
2	0,60	3,5406	1,0056	-0,0014	0,0027	-0,0060		
3	0,65	4,5462	1,0042	0,0013	-0,0033			
4	0,70	5,5504	1,0055	-0,0020				
5	0,75	6,5559	1,0035					
6	0,80	7,5594						

1.3. Вычисление значения функции для $X_1 = 0,502$

Так как точка $X_1 = 0,502$ находится в начале таблицы (ближе к $x_0 = 0,50$), используем **Первую интерполяционную формулу Ньютона (вперед)**:

$$N_1(x) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_0 + \frac{t(t-1)(t-2)}{3!}\Delta^3 y_0 + \dots$$

Вычислим t :

$$t = \frac{x - x_0}{h} = \frac{0,502 - 0,50}{0,05} = \frac{0,002}{0,05} = 0,04$$

Подставляем значения из таблицы (ограничимся первыми 4 слагаемыми для ручного счета):

$$\begin{aligned} N_1(0,502) &\approx 1,5320 + 0,04 \cdot 1,0036 + \frac{0,04 \cdot (-0,96)}{2} \cdot 0,0014 + \frac{0,04 \cdot (-0,96) \cdot (-1,96)}{6} \cdot (-0,0008) = \\ &= 1,5320 + 0,040144 - 0,00002688 - 0,000010035 = \mathbf{1,572107} \end{aligned}$$

1.4. Вычисление значения функции для $X_2 = 0,645$

Точка $X_2 = 0,645$ находится в середине таблицы. Выберем в качестве центрального узла $x_3 = 0,65$. Так как $x < x_3$, применяем **Вторую интерполяционную формулу Гаусса (назад)**:

$$G_2(x) = y_0 + t\Delta y_{-1} + \frac{t(t+1)}{2!}\Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!}\Delta^3 y_{-2} + \dots$$

Где y_0 в формуле соответствует нашему $y_3 = 4,5462$. Следовательно: $\Delta y_{-1} \rightarrow \Delta y_2 = 1,0056$; $\Delta^2 y_{-1} \rightarrow \Delta^2 y_2 = -0,0014$; $\Delta^3 y_{-2} \rightarrow \Delta^3 y_1 = -0,0020$.

Вычислим t :

$$t = \frac{x - x_3}{h} = \frac{0,645 - 0,65}{0,05} = -0,1$$

Подставляем:

$$\begin{aligned} G_2(0, 645) &\approx 4, 5462 + (-0, 1) \cdot 1, 0056 + \frac{-0, 1 \cdot 0, 9}{2} \cdot (-0, 0014) + \frac{0, 9 \cdot (-0, 1) \cdot (-1, 1)}{6} \cdot (-0, 0020) = \\ &= 4, 5462 - 0, 10056 + 0, 000063 - 0, 000033 = \mathbf{4, 44567} \end{aligned}$$

1.5. Вычисление значений через многочлен Лагранжа

Общая формула многочлена Лагранжа:

$$L_n(x) = \sum_{i=0}^n y_i \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

Для получения высокой точности программа использует все 7 узлов интерполяции. Для ручной демонстрации приведем базисный полином $l_0(x)$ для первой точки:

$$l_0(x) = \frac{(x - 0, 55)(x - 0, 60)(x - 0, 65)(x - 0, 70)(x - 0, 75)(x - 0, 80)}{(0, 50 - 0, 55)(0, 50 - 0, 60)(0, 50 - 0, 65)(0, 50 - 0, 70)(0, 50 - 0, 75)(0, 50 - 0, 80)}$$

Итоговые расчеты, выполненные программно по всем 7 точкам, дают следующие значения:

- $L(0, 502) \approx \mathbf{1, 57211}$
- $L(0, 645) \approx \mathbf{4, 44567}$

Как видно, результаты вычислений методами конечных разностей и методом Лагранжа совпадают с высокой точностью.

2. Листинг программы

```
1  #include <iostream>
2  #include <vector>
3  #include <cmath>
4  #include <iomanip>
5  #include <string>
6  #include <limits>
7  #include <fstream>
8  #include <algorithm>
9  #include "matplotlibcpp.h"
10
11 namespace plt = matplotlibcpp;
12 using namespace std;
13
14 struct Point {
15     double x;
16     double y;
17 };
18
19 template <typename T>
20 T get_input(const string& prompt) {
21     T value;
22     while (true) {
23         cout << prompt;
24         if (cin >> value) {
25             if (cin.peek() == '\n' || cin.peek() == ' ') {
26                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
27                 return value;
28             }
29         }
30         cin.clear();
31         cin.ignore(numeric_limits<streamsize>::max(), '\n');
32         cout << "Введено некорректное значение\n";
33     }
34 }
35
36 vector<vector<double>> build_diff_table(const vector<Point>& pts) {
37     int n = pts.size();
38     vector<vector<double>> diff(n, vector<double>(n, 0.0));
39     for (int i = 0; i < n; i++) {
40         diff[i][0] = pts[i].y;
41     }
42     for (int j = 1; j < n; j++) {
43         for (int i = 0; i < n - j; i++) {
44             diff[i][j] = diff[i + 1][j - 1] - diff[i][j - 1];
45         }
46     }
47     return diff;
48 }
49
50 void print_diff_table(const vector<Point>& pts, const vector<vector<double>>& diff) {
51     int n = pts.size();
52     cout << "\nТаблица конечных разностей:\n";
53     cout << setw(10) << "x" << setw(15) << "y";
54     for (int j = 1; j < n; j++) {
55         cout << setw(15) << "d^" + to_string(j) + "y";
56     }
57     cout << "\n";
```

```

58     for (int i = 0; i < n; i++) {
59         cout << setw(10) << fixed << setprecision(5) << pts[i].x;
60         for (int j = 0; j < n - i; j++) {
61             cout << setw(15) << fixed << setprecision(5) << diff[i][j];
62         }
63         cout << "\n";
64     }
65     cout << "\n";
66 }
67
68 double lagrange(const vector<Point>& pts, double x) {
69     double res = 0;
70     int n = pts.size();
71     for (int i = 0; i < n; i++) {
72         double term = pts[i].y;
73         for (int j = 0; j < n; j++) {
74             if (i != j) {
75                 term *= (x - pts[j].x) / (pts[i].x - pts[j].x);
76             }
77         }
78         res += term;
79     }
80     return res;
81 }
82
83 double newton_forward(const vector<Point>& pts, const vector<vector<double>>& diff,
84 ↪ double x) {
85     int n = pts.size();
86     double h = pts[1].x - pts[0].x;
87     double t = (x - pts[0].x) / h;
88     double res = diff[0][0];
89     double term = 1;
90     for (int i = 1; i < n; i++) {
91         term *= (t - i + 1) / i;
92         res += term * diff[0][i];
93     }
94     return res;
95 }
96
97 double newton_backward(const vector<Point>& pts, const vector<vector<double>>& diff,
98 ↪ double x) {
99     int n = pts.size();
100     double h = pts[1].x - pts[0].x;
101     double t = (x - pts[n - 1].x) / h;
102     double res = diff[n - 1][0];
103     double term = 1;
104     for (int i = 1; i < n; i++) {
105         term *= (t + i - 1) / i;
106         res += term * diff[n - 1 - i][i];
107     }
108     return res;
109 }
110
111 double gauss_forward(const vector<Point>& pts, const vector<vector<double>>& diff,
112 ↪ double x) {
113     int n = pts.size();
114     double h = pts[1].x - pts[0].x;
115     int k = 0;
116     double min_dist = abs(x - pts[0].x);
117     for (int i = 1; i < n; i++) {
118         if (abs(x - pts[i].x) < min_dist) {

```

```

116         min_dist = abs(x - pts[i].x);
117         k = i;
118     }
119 }
120 if (x < pts[k].x && k > 0) k--;
121
122 double t = (x - pts[k].x) / h;
123 double res = diff[k][0];
124 double term = 1;
125 int cur_idx = k;
126
127 for (int i = 1; i < n; i++) {
128     if (i % 2 == 1) {
129         term *= (t - i / 2.0) / i;
130     } else {
131         term *= (t + i / 2.0) / i;
132         cur_idx--;
133     }
134     if (cur_idx < 0 || cur_idx + i >= n) break;
135     res += term * diff[cur_idx][i];
136 }
137 return res;
138 }
139
140 double gauss_backward(const vector<Point>& pts, const vector<vector<double>>& diff,
141 ↪ double x) {
142     int n = pts.size();
143     double h = pts[1].x - pts[0].x;
144     int k = 0;
145     double min_dist = abs(x - pts[0].x);
146     for (int i = 1; i < n; i++) {
147         if (abs(x - pts[i].x) < min_dist) {
148             min_dist = abs(x - pts[i].x);
149             k = i;
150         }
151     }
152     if (x > pts[k].x && k < n - 1) k++;
153
154     double t = (x - pts[k].x) / h;
155     double res = diff[k][0];
156     double term = 1;
157     int cur_idx = k;
158
159     for (int i = 1; i < n; i++) {
160         if (i % 2 == 1) {
161             term *= (t + i / 2.0) / i;
162             cur_idx--;
163         } else {
164             term *= (t - i / 2.0) / i;
165         }
166         if (cur_idx < 0 || cur_idx + i >= n) break;
167         res += term * diff[cur_idx][i];
168     }
169     return res;
170 }
171
172 double stirling(const vector<Point>& pts, const vector<vector<double>>& diff, double x)
173 ↪ {
174     return (gauss_forward(pts, diff, x) + gauss_backward(pts, diff, x)) / 2.0;
175 }

```

```

175 double bessell(const vector<Point>& pts, const vector<vector<double>>& diff, double x) {
176     int n = pts.size();
177     double h = pts[1].x - pts[0].x;
178     int k = 0;
179     for (int i = 0; i < n - 1; i++) {
180         if (x >= pts[i].x && x <= pts[i + 1].x) {
181             k = i;
182             break;
183         }
184     }
185
186     double t = (x - pts[k].x) / h;
187     double res = (diff[k][0] + diff[k + 1][0]) / 2.0;
188     res += (t - 0.5) * diff[k][1];
189
190     double term = 1;
191     int cur_idx = k;
192     for (int i = 2; i < n; i++) {
193         if (i % 2 == 0) {
194             term *= (t - i / 2.0 + 1) * (t + i / 2.0 - 1) / (i * (i - 1.0));
195             if (cur_idx - 1 < 0 || cur_idx + i - 1 >= n) break;
196             res += term * (diff[cur_idx - 1][i] + diff[cur_idx][i]) / 2.0;
197             cur_idx--;
198         } else {
199             term *= (t - 0.5) / i;
200             if (cur_idx < 0 || cur_idx + i >= n) break;
201             res += term * diff[cur_idx][i];
202         }
203     }
204     return res;
205 }
206
207 double eval_best_method(const vector<Point>& pts, const vector<vector<double>>& diff,
↪ double x) {
208     int n = pts.size();
209     if (x <= pts[n / 2].x) return newton_forward(pts, diff, x);
210     else return newton_backward(pts, diff, x);
211 }
212
213 void plot_interpolation(const vector<Point>& pts, const vector<vector<double>>& diff,
↪ double target_x, double target_y) {
214     vector<double> x_pts, y_pts;
215     double min_x = pts[0].x;
216     double max_x = pts.back().x;
217
218     for (const auto& p : pts) {
219         x_pts.push_back(p.x);
220         y_pts.push_back(p.y);
221     }
222
223     int N = 100;
224     double padding = (max_x - min_x) * 0.1;
225     double step = (max_x - min_x + 2 * padding) / (N - 1);
226     vector<double> curve_x, curve_y;
227
228     for (int i = 0; i < N; i++) {
229         double x = min_x - padding + i * step;
230         curve_x.push_back(x);
231         curve_y.push_back(eval_best_method(pts, diff, x));
232     }
233

```



```

234 plt::figure();
235 plt::scatter(x_pts, y_pts, 30.0, {"color", "red"}, {"label", "Узлы
↳ интерполяции"}));
236 plt::plot(curve_x, curve_y, {"color", "blue"}, {"label", "Интерполяционный
↳ многочлен"}));
237 plt::scatter(vector<double>{target_x}, vector<double>{target_y}, 50.0, {"color",
↳ "green"}, {"label", "Искомая точка"}));
238
239 plt::xlabel("X");
240 plt::ylabel("Y");
241 plt::title("Интерполяция функции");
242 plt::legend();
243 plt::grid(true);
244 plt::show();
245 }
246
247 int main() {
248     vector<Point> points;
249
250     int input_type;
251     while(true) {
252         cout << "Выбор ввода данных:\n1 - С клавиатуры\n2 - Из файла\n3 - Выбор
↳ функции\nВвод: ";
253         input_type = get_input<int>("");
254         if(input_type >= 1 && input_type <= 3) break;
255         cout << "Введите 1, 2 или 3\n";
256     }
257
258     auto diff_table = build_diff_table(points);
259     print_diff_table(points, diff_table);
260
261     while (true) {
262         double x = get_input<double>("Введите значение аргумента X для интерполяции: ");
263
264         cout << "\nРезультаты интерполяции:\n";
265         cout << string(50, '-') << "\n";
266
267         double res_lagrange = lagrange(points, x);
268         cout << left << setw(30) << "Многочлен Лагранжа:" << res_lagrange << "\n";
269
270         double res_nf = newton_forward(points, diff_table, x);
271         double res_nb = newton_backward(points, diff_table, x);
272         double mid_x = (points[0].x + points.back().x) / 2.0;
273
274         if (x <= mid_x) {
275             cout << left << setw(30) << "Многочлен Ньютона (вперед):" << res_nf << "\n";
276         } else {
277             cout << left << setw(30) << "Многочлен Ньютона (назад):" << res_nb << "\n";
278         }
279
280         double res_gf = gauss_forward(points, diff_table, x);
281         double res_gb = gauss_backward(points, diff_table, x);
282
283         cout << left << setw(30) << "Многочлен Гаусса (вперед):" << res_gf << "\n";
284         cout << left << setw(30) << "Многочлен Гаусса (назад):" << res_gb << "\n";
285
286         double res_stirling = stirling(points, diff_table, x);
287         cout << left << setw(30) << "Схема Стирлинга:" << res_stirling << "\n";
288
289         double res_bessel = bessel(points, diff_table, x);
290         cout << left << setw(30) << "Схема Бесселя:" << res_bessel << "\n";

```

```

291     cout << string(50, '-') << "\n";
292
293     double final_res = eval_best_method(points, diff_table, x);
294     plot_interpolation(points, diff_table, x, final_res);
295
296     int cont;
297     while (true) {
298         cout << "Хотите проверить другой X? (1 - Да, 0 - Нет): ";
299         cont = get_input<int>("");
300         if (cont == 0 || cont == 1) break;
301     }
302     if (cont == 0) break;
303 }
304
305
306     return 0;
307 }

```

3. Вывод

В ходе выполнения данной лабораторной работы я рассмотрел и реализовал методы интерполяции Ньютона и Гаусса для заданной таблицы данных. Результаты показали, что оба метода могут быть эффективно использованы для интерполяции, но их точность может зависеть от конкретной функции и распределения данных.